

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

Taller de diseño digital

Carlos Steve Alvarado Mendez
Kevin Antonio Carranza Blanco

Propuestas de Diseño para Calculadora ALU - Laboratorio 2

Análisis del Problema

Se requiere diseñar una calculadora parametrizable que ejecute las siguientes operaciones:

- **Aritméticas:** suma, resta, multiplicación, división, módulo
- **Lógicas:** AND, OR, XOR
- **Desplazamiento:** shift left, shift right

Con banderas de estado: Negativo (N), Cero (Z), Acarreo (C) y Desbordamiento (V).

Propuesta 1: Diseño Modular con Control Centralizado

Características Principales

- **Arquitectura:** Modular con unidades especializadas
- **Control:** Centralizado en `control_unit`
- **Interfaz:** Dual mode (configuración/operación)

Diagrama de Bloques - Propuesta 1

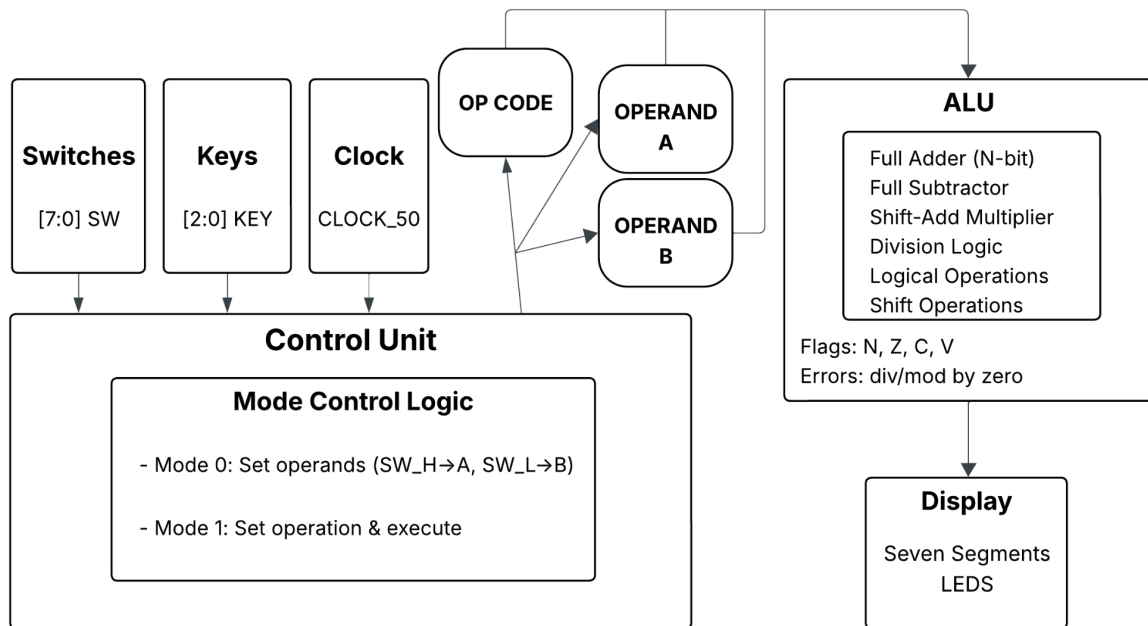


Tabla de Verdad para Control de Operaciones

OpCode	Operación	Descripción
0000	ADD	$A + B$
0001	SUB	$A - B$
0010	MUL	$A \times B$ (N bits bajos)
0011	DIV	$A \div B$
0100	MOD	$A \% B$
0101	AND	$A \& B$
0110	OR	$A B$
0111	XOR	$A \wedge B$
1000	SHL	$A \ll B$
1001	SHR	$A \gg B$

Ventajas

- **Modularidad:** Cada operación tiene su módulo dedicado
- **Escalabilidad:** Fácil agregar nuevas operaciones
- **Claridad:** Separación clara entre control y procesamiento
- **Reutilización:** Módulos pueden reutilizarse en otros diseños

Desventajas

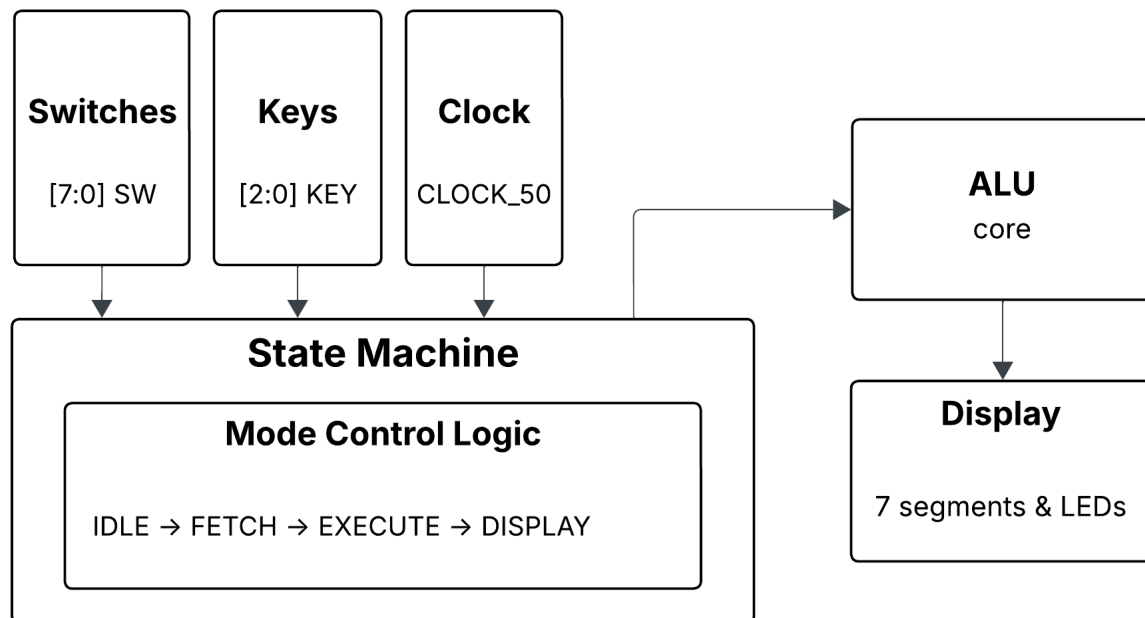
- **Recursos:** Mayor uso de recursos FPGA
- **Complejidad:** Múltiples módulos aumentan complejidad de debugging
- **Latencia:** Múltiples niveles de lógica pueden aumentar delay

Propuesta 2: Diseño Pipeline con Máquina de Estados

Características Principales

- **Arquitectura:** Pipeline de 3 etapas
- **Control:** Máquina de estados finitos
- **Interfaz:** Secuencial por etapas

Diagrama de Bloques - Propuesta 2



Máquina de Estados

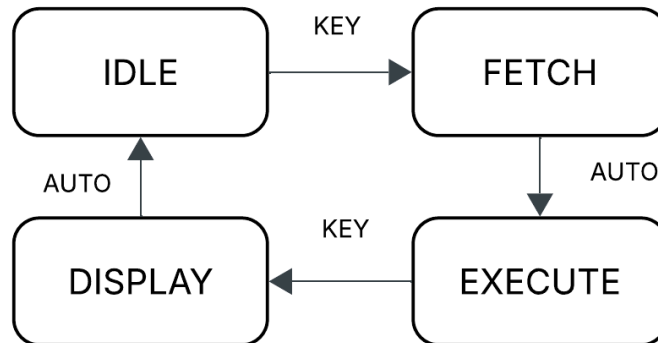


Tabla de Estados

Estado	Entradas Activas	Acción	Siguiente Estado
IDLE	KEY0	Cargar operandos A,B de SW	FETCH
FETCH	Auto	Cargar operación de SW[3:0]	EXECUTE
EXECUTE	Auto	Ejecutar operación en ALU	DISPLAY
DISPLAY	KEY1	Mostrar resultado	IDLE

ALU Core Simplificado

// Diseño con operadores directos para la mayoría de operaciones

```
always_comb begin
```

```
  case(opcode)
```

```
    4'b0000: result = add_unit(a, b);    // Módulo dedicado
```

```
    4'b0001: result = sub_unit(a, b);    // Módulo dedicado
```

```
    4'b0010: result = mult_unit(a, b);   // Módulo dedicado
```

```
    default: result = direct_ops(a, b, opcode); // Operadores HDL
```

```
  endcase
```

```
end
```

Ventajas

- **Velocidad:** Pipeline permite mayor throughput
- **Recursos:** Menor uso de recursos al reutilizar lógica
- **Timing:** Mejor control de timing con estados definidos
- **Flexibilidad:** Fácil modificar secuencia de operación

Desventajas

- **Complejidad:** Máquina de estados más compleja
- **Latencia:** Múltiples ciclos por operación
- **Debug:** Más difícil debuggear problemas de timing

Comparación de Propuestas

Aspecto	Propuesta 1 (Modular)	Propuesta 2 (Pipeline)
Recursos FPGA	Alto (múltiples módulos)	Medio (lógica compartida)
Velocidad	Alta (combinacional)	Media (secuencial)
Complejidad	Media	Alta
Escalabilidad	Excelente	Buena
Debug	Fácil	Complejo
Timing	Crítico	Controlado

Propuesta elegida

Se eligió la **Propuesta 1** (Diseño Modular) por las siguientes razones:

1. **Cumple mejor con los requisitos:** El laboratorio pide diseño estructural partiendo de circuitos básicos
2. **Mejor para aprendizaje:** Permite entender cada operación individualmente
3. **Facilidad de testing:** Cada módulo se puede probar independientemente